

PEARLS AQI PREDICTOR

**End-to-End Air Quality Index Forecasting
System for Peshawar**

PROJECT REPORT

Prepared By:

Iqra Jan

September 6th, 2026

Contents

1. Executive Summary	5
2. Problem Statement.....	6
3. System Architecture Overview	6
4. Data Collection	7
4.1 Data Source.....	7
5. Weather Data	8
6. Air-Quality Data	8
7. Historical Data Backfill	9
8. Data Processing	9
9. Feature Engineering	10
10. Model Features.....	10
10.1 Weather Features.....	10
10.2 Pollutant Features	11
10.3 AQI Lag Features	11
11. Pollutant Lag Features	11
12. Rolling Features	12
13. Change Features.....	12
14. Time-Based Features	13
15. Target Variable	13
16. Data Leakage Prevention	13
17. Feast Feature Store	14
18. PostgreSQL and Supabase	14
19. Model Development	15
20. Training, Validation, and Testing	15
21. Model Comparison.....	16
22. Champion Model.....	16
23. MLflow Model Registry	16
24. Forecasting Methodology	17
25. Recursive Forecasting.....	17
26. Daily Forecast.....	17
27. 72-Hour Evaluation	17

28. Persistence Baseline	18
29. FastAPI Backend	18
30. API Endpoints	19
GET /health	19
GET /current	19
GET /current/trend.....	19
GET /forecast	19
GET /shap/{forecast_day}	19
31. Streamlit Dashboard.....	19
32. Dashboard Features.....	20
33. Daily SHAP Aggregation.....	20
34. CI/CD Architecture	20
34.1 CI/CD Scheduling Limitation	21
35. Feature Pipeline	21
36. Training Pipeline.....	21
37. Project Structure.....	22
38. Separation Between Development and Production.....	23
39. Validation and Testing	23
Data validation	24
Feature validation	24
Model validation.....	24
Application validation	24
40. Limitations.....	24
40.1 Location limitation.....	25
40.2 Forecast uncertainty	25
40.3 Recursive error.....	25
40.4 External data dependency	25
40.5 Distribution shift	25
41. Future Improvements.....	25
1. Prediction Intervals.....	25
2. Model Drift Monitoring.....	25
3. Multi-City Forecasting	26
4. Automated Retraining	26

5. Better Monitoring	26
42. Complete Technology Stack.....	26
43. Final Conclusion.....	27
44. Project Summary	28

1. Executive Summary

Pearls AQI Predictor is an end-to-end machine learning system developed to monitor air quality in Peshawar and predict the Air Quality Index (AQI) for the next three days.

It implements a complete machine learning pipeline starting from automated data collection and historical data backfilling, followed by data processing, feature engineering, feature-store management, model training, model evaluation, model registration, recursive forecasting, API serving, visualization, explainability, and CI/CD automation.

The system uses Open-Meteo as the primary source for historical and forecast weather and air-quality data. Approximately two years of hourly historical data were collected, covering the period from August 1, 2024 to August 1, 2026.

The final merged historical dataset contains 17,544 hourly records and 20 columns. The data was validated for duplicate timestamps and missing values. After feature engineering and removal of rows that could not be used because of lag/rolling requirements and target shifting, the processed training dataset contains 17,471 rows and 79 columns.

Exactly 70 features are used as model inputs. These features include current weather measurements, pollutant concentrations, AQI lag variables, pollutant lag variables, rolling averages, change features, and temporal/calendar features.

The prediction target is the AQI for the following hour:

target_aqi = us_aqi.shift(-1)

Three machine learning models were trained and evaluated:

1. Ridge Regression
2. Random Forest Regression
3. XGBoost Regression

The models were evaluated using a chronological train/validation/test split rather than random shuffling. XGBoost produced the best test results with:

- **MAE:** 1.3132
- **RMSE:** 2.4022
- **R²:** 0.9943

Therefore, XGBoost was selected as the champion model.

The trained champion model is registered through MLflow and identified using the model name:

Pearls_AQI_XGBoost

with the production alias:

champion

The forecasting system generates 72 hourly predictions recursively. The latest observed AQI is used to initialize the forecast. After the first prediction, the predicted AQI becomes the AQI state used for the following prediction. This prevents the system from using future actual AQI values that would not be available in a real deployment.

The prediction service is exposed through FastAPI, while the Streamlit dashboard provides the user interface. The dashboard displays current AQI, recent AQI trends, three-day forecasts, model information, evaluation metrics, and SHAP-based explanations.

GitHub Actions automates the data and machine learning lifecycle. The feature pipeline runs hourly, while the training pipeline runs daily after successful feature processing and can also be manually triggered.

2. Problem Statement

Air pollution is highly dynamic and changes according to pollutant concentrations, weather conditions, seasonal patterns, and recent air-quality conditions.

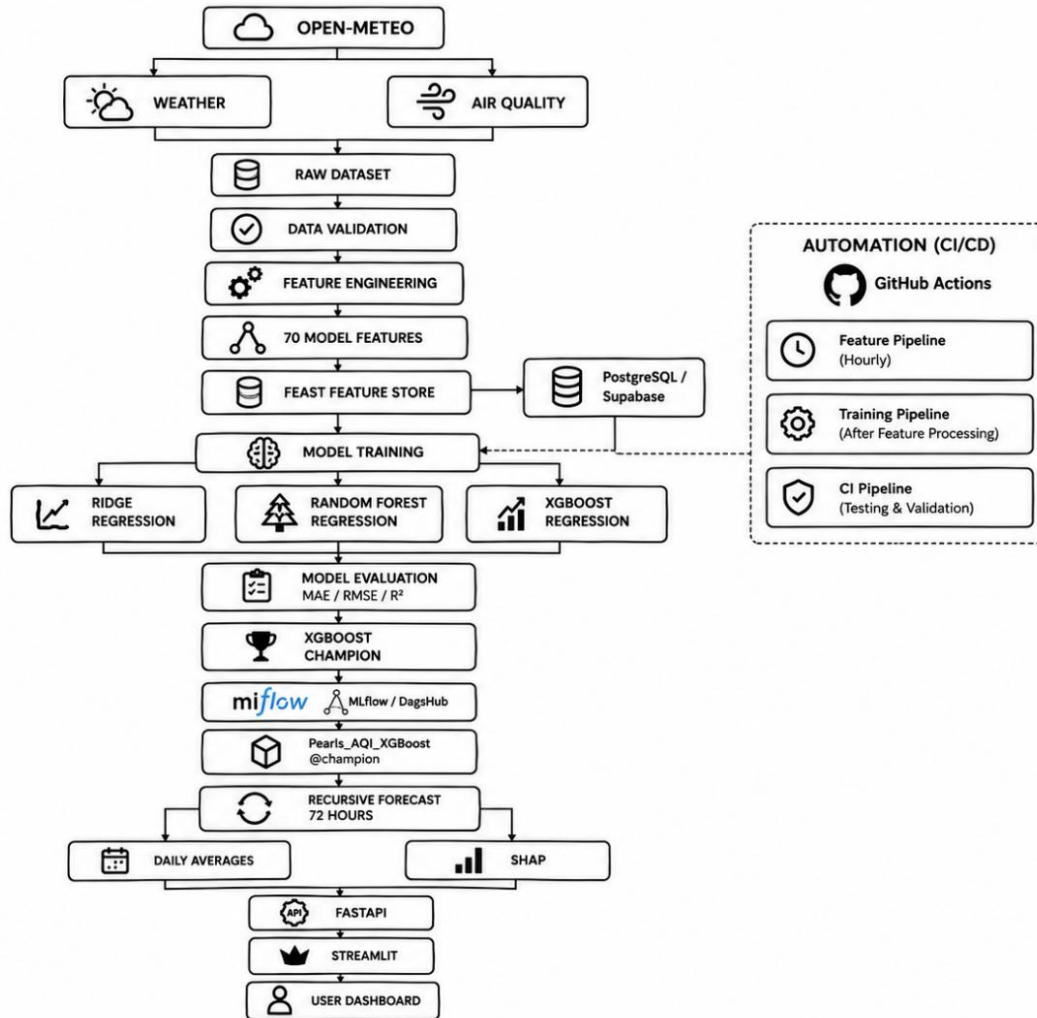
A simple current AQI measurement only describes the present state of air quality. It does not provide information about what air quality may look like several hours or days into the future.

The purpose of this project is therefore to build a machine learning system capable of learning historical relationships between AQI, pollutants, weather, and temporal patterns and using those relationships to predict future AQI.

The system focuses on Peshawar and produces a three-day forecast so that users can understand expected air-quality conditions in advance.

3. System Architecture Overview

The complete system follows this general architecture:



4. Data Collection

4.1 Data Source

The project uses Open-Meteo as its main data source.

Open-Meteo provides both:

- Weather information
- Air-quality information

This makes it possible to obtain the variables required by the forecasting system without depending on multiple unrelated data providers.

The project location is Peshawar, Pakistan.

Approximate coordinates:

Latitude: 34.008° N

Longitude: 71.5785° E

5. Weather Data

The weather data contains hourly environmental variables that can influence air-quality conditions.

The main weather variables include:

- Temperature
- Relative humidity
- Mean sea-level pressure
- Precipitation
- Wind speed
- Wind direction

6. Air-Quality Data

The air-quality data contains pollutant measurements and AQI-related variables.

The main pollutant variables include:

- PM2.5
- PM10
- Carbon monoxide
- Nitrogen dioxide
- Sulphur dioxide
- Ozone

The dataset also contains:

- US AQI
- AQI associated with PM2.5
- AQI associated with PM10
- AQI associated with carbon monoxide
- AQI associated with nitrogen dioxide
- AQI associated with sulphur dioxide

- AQI associated with ozone

7. Historical Data Backfill

A major requirement of the project was obtaining enough historical data for reliable time-series model training.

The final historical backfill covered approximately two years:

Start: August 1, 2024

End: August 1, 2026

The historical data was collected at hourly frequency.

The final merged raw dataset contained:

17,544 rows × 20 columns

Validation showed:

- Duplicate timestamps: 0
- Missing values: none in the validated merged dataset

The historical backfill was important because it provided enough information to learn:

- Daily patterns
- Seasonal patterns
- AQI persistence
- Pollutant behavior
- Weather relationships
- Short-term changes
- Long-term rolling behavior

8. Data Processing

After collection, weather and air-quality datasets were merged using their timestamps.

The processing pipeline performs several validation steps.

These include:

- Timestamp validation

- Duplicate timestamp detection
- Missing-value detection
- Chronological ordering
- Expected column validation
- Data consistency checks

The data is maintained at hourly frequency because the prediction target is the next-hour AQI.

9. Feature Engineering

Feature engineering is one of the most important components of the project.

The raw measurements alone do not provide enough historical context for the forecasting model.

Therefore, historical lag features, rolling features, change features, and temporal features were created.

The final processed dataset contains:

17,471 rows × 79 columns

Of these:

70 columns are model features.

10. Model Features

The 70 model features can be grouped into several categories.

10.1 Weather Features

Weather features include:

- temperature_2m
- relative_humidity_2m
- pressure_msl
- precipitation
- wind_speed_10m
- wind_direction_10m

10.2 Pollutant Features

Pollutant features include:

- pm2_5
- pm10
- carbon_monoxide
- nitrogen_dioxide
- sulphur_dioxide
- ozone

10.3 AQI Lag Features

Historical AQI values are used to capture temporal persistence.

The project includes AQI lag variables such as:

- AQI 1 hour ago
- AQI 3 hours ago
- AQI 6 hours ago
- AQI 12 hours ago
- AQI 24 hours ago
- AQI 48 hours ago
- AQI 72 hours ago

These features allow the model to understand how recent AQI affects future AQI.

11. Pollutant Lag Features

Historical pollutant measurements are also included through lag features.

Lag variables are created for pollutants such as:

- PM2.5
- PM10
- CO
- NO₂
- SO₂
- O₃

This allows the model to understand whether recent pollutant levels are associated with future AQI.

12. Rolling Features

Rolling statistics are used to represent recent pollution conditions.

AQI rolling means include:

- 3-hour mean
- 6-hour mean
- 12-hour mean
- 24-hour mean

PM2.5 and PM10 rolling means include:

- 3-hour mean
- 6-hour mean
- 24-hour mean

The system also calculates 24-hour rolling means for:

- Carbon monoxide
- Nitrogen dioxide
- Sulphur dioxide
- Ozone

Rolling features reduce the effect of isolated hourly fluctuations and provide the model with information about recent pollution trends.

13. Change Features

The project also uses AQI and pollutant change features.

These represent how rapidly pollution is changing.

Examples include:

- AQI change over 1 hour
- AQI change over 3 hours
- AQI change over 6 hours
- Changes in pollutant concentrations

These features help the model distinguish between a stable AQI condition and a rapidly increasing or decreasing AQI condition.

14. Time-Based Features

AQI patterns can vary depending on the time of day and calendar conditions.

The project therefore uses temporal features such as:

- Hour
- Day
- Month
- Day of week
- Weekend-related information

These features allow the model to learn recurring temporal patterns.

15. Target Variable

The target variable represents the next-hour AQI.

It is created as:

```
target_aqi = us_aqi.shift(-1)
```

Therefore, if the current observation is at time T , the target corresponds to AQI at $T+1$ hour.

This converts the historical AQI dataset into a supervised learning problem.

16. Data Leakage Prevention

Data leakage is a major risk in time-series machine learning.

The project uses several mechanisms to prevent leakage.

First, the data is split chronologically rather than randomly.

Second, rolling features are calculated using historical information rather than future observations.

Third, the target variable is not used as a model input.

Fourth, the recursive prediction system does not use future actual AQI values.

The model receives exactly 70 model features.

The latest AQI is maintained separately as the recursive prediction state.

17. Feast Feature Store

Feast is used as the feature-store layer.

The main feature view is:

aqi_features

The entity is:

location_id

The feature store provides a consistent mechanism for accessing the features required by the machine learning system.

The project contains:

- Feature view
- Training feature service
- Prediction context
- Online feature serving

The training feature service is:

aqi_model_features

The prediction context is:

aqi_prediction_context

The feature-store TTL is configured for approximately seven days.

18. PostgreSQL and Supabase

PostgreSQL is used as the cloud database infrastructure for the Feast configuration.

Supabase provides the managed PostgreSQL environment.

This gives the system persistent cloud infrastructure rather than relying only on a local feature-store database.

The project also maintains local Feast configuration for development and testing.

This creates two environments:

Local environment → development/testing

Cloud environment → production-oriented feature serving

19. Model Development

Three regression models were developed and compared.

The models are:

1. Ridge Regression
2. Random Forest Regression
3. XGBoost Regression

All models were trained using the engineered AQI feature dataset.

20. Training, Validation, and Testing

The processed dataset contains 17,471 usable rows.

The data was split chronologically:

<i>Dataset</i>	<i>Rows</i>	<i>Percentage</i>
<i>Training</i>	12,229	70%
<i>Validation</i>	2,621	15%
<i>Testing</i>	2,621	15%
<i>Total</i>	17,471	100%

Chronological splitting is particularly important for AQI forecasting because randomly mixing past and future observations could produce unrealistic evaluation results.

21. Model Comparison

The final test results are:

<i>Model</i>	<i>MAE</i>	<i>RMSE</i>	<i>R²</i>
<i>Ridge Regression</i>	2.7992	4.3062	0.9816
<i>Random Forest</i>	2.6022	4.4868	0.9800
<i>XGBoost</i>	1.3132	2.4022	0.9943

XGBoost achieved the lowest MAE and RMSE and the highest R².

Therefore:

XGBoost was selected as the champion model.

22. Champion Model

The production model is: **Pearls_AQI_XGBoost**

The production alias is: **champion**

The model can be referenced using: **models:/Pearls_AQI_XGBoost@champion**

A newly validated model can be registered and assigned the champion alias.

The prediction service then loads the champion automatically.

23. MLflow Model Registry

MLflow is used for:

- Experiment tracking
- Model logging
- Model versioning
- Model registry
- Production model selection
- Model metadata
- Evaluation metric storage

DagsHub provides the cloud MLflow tracking and registry layer.

24. Forecasting Methodology

The production system generates: **72 hourly AQI predictions**

representing: **3 days × 24 hours**

The system uses: **HISTORY_HOURS = 96**

and: **FORECAST_HOURS = 72**

The 96-hour historical context gives the forecasting pipeline enough recent information to calculate the required lag and rolling features.

25. Recursive Forecasting

The forecast is generated recursively.

The process is:

1. Obtain the latest observed AQI.
2. Construct the first future feature vector.
3. Send the 70 model features to the XGBoost champion model.
4. Generate the first future AQI prediction.
5. Store the prediction.
6. Use that prediction as the AQI state for the next prediction.
7. Update lag and rolling information.
8. Generate the next prediction.
9. Continue until all 72 hours have been predicted.

26. Daily Forecast

The 72 hourly predictions are aggregated into three daily averages.

The dashboard therefore presents:

- Day 1 average AQI
- Day 2 average AQI
- Day 3 average AQI

27. 72-Hour Evaluation

The project includes a dedicated 72-hour evaluation pipeline.

The evaluation records:

- predicted_aqi
- actual_aqi
- persistence_aqi

It calculates:

- RMSE
- MAE
- R^2

The evaluation can also provide separate performance information for:

- Day 1
- Day 2
- Day 3

This is important because a model may perform well one hour ahead but accumulate more error at longer forecast horizons.

28. Persistence Baseline

The evaluation pipeline also keeps a persistence forecast.

Persistence means that the most recent AQI value is carried forward as the prediction.

This is a simple but useful baseline.

A machine learning model should ideally provide better performance than simply assuming that AQI will remain unchanged.

29. FastAPI Backend

FastAPI is used as the backend API layer.

The backend provides access to:

- Current AQI
- Current AQI trend

- Forecast
- Model information
- SHAP explanations
- Health information

The API provides a separation between the machine learning infrastructure and the user interface.

30. API Endpoints

The main endpoints include:

GET /health

Used to check whether the backend service is operating correctly.

GET /current

Returns the current AQI and related information.

GET /current/trend

Returns recent AQI trend information.

GET /forecast

Returns the three-day forecast and model information.

GET /shap/{forecast_day}

Returns SHAP explanations for the selected forecast day.

31. Streamlit Dashboard

Streamlit is used to create the user-facing dashboard.

The dashboard does not directly bypass the production architecture.

Instead:

Streamlit → FastAPI → prediction/ML infrastructure

This keeps the frontend separate from the backend.

The dashboard is designed for Peshawar AQI monitoring and forecasting.

32. Dashboard Features

The dashboard includes:

- Current AQI
- AQI category
- Current timestamp
- Recent AQI trend
- Three-day AQI forecast
- Daily forecast cards
- Model information
- Model metrics
- Forecast evaluation metrics
- SHAP explanations
- Refresh functionality

The dashboard converts timestamps into Pakistan Standard Time for user-friendly presentation.

33. Daily SHAP Aggregation

The forecasting system produces hourly predictions.

Therefore, SHAP explanations can also be produced for individual forecast hours.

The dashboard aggregates approximately 24 hourly explanations for each forecast day.

This provides a simpler daily explanation instead of overwhelming the user with 24 separate explanations.

34. CI/CD Architecture

GitHub Actions automates the project's machine learning lifecycle.

The project contains separate workflows for:

- Hourly Feature pipeline
- Daily Training pipeline

34.1 CI/CD Scheduling Limitation

The feature pipeline is configured in GitHub Actions to execute automatically on an hourly schedule. However, scheduled GitHub Actions workflows are not guaranteed to start at the exact scheduled time. Due to GitHub's server-side scheduling and runner availability, scheduled execution may be delayed or vary from the configured time.

Therefore, although the system is designed for hourly automated feature updates, the actual execution time can vary. This limitation is related to the GitHub Actions infrastructure rather than the project's workflow logic. The pipeline also supports manual execution, which provides an alternative when an immediate feature update is required.

For a production environment requiring strict hourly execution, a dedicated scheduler or managed orchestration service with guaranteed scheduling would provide more predictable execution.

35. Feature Pipeline

The feature pipeline is designed to run hourly.

The configured schedule is approximately:

```
**15 * * * **
```

This means that the workflow runs around minute 15 of each hour.

The pipeline performs steps such as:

1. Update raw data.
2. Validate the raw dataset.
3. Run feature engineering.
4. Validate the processed dataset.
5. Verify required columns.
6. Confirm that the feature dataset is ready for downstream processing.

36. Training Pipeline

The training pipeline is designed to run after successful feature processing and can also be manually triggered.

The training workflow:

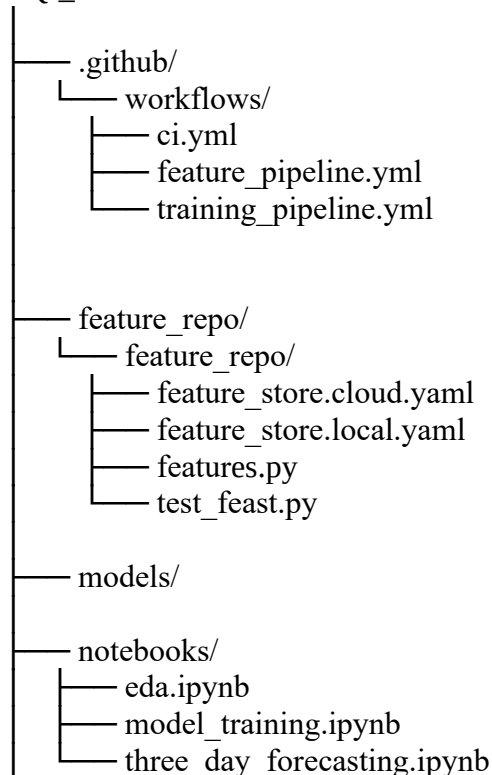
1. Installs dependencies.
2. Checks required Feast dependencies.
3. Loads required secrets.
4. Configures cloud Feast.
5. Configures MLflow credentials.
6. Verifies that MLflow is using a cloud tracking location.
7. Runs model training.
8. Evaluates the models.
9. Registers the model.
10. Verifies the XGBoost champion alias.

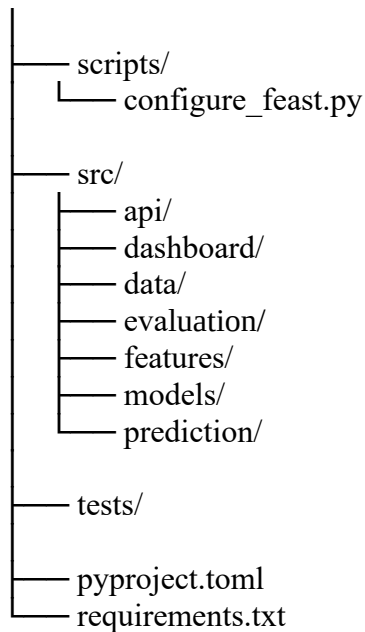
This creates an automated model lifecycle.

37. Project Structure

The project repository is organized as follows:

AQI_Predictor/





38. Separation Between Development and Production

A major design principle of the project is separating experimentation from production.

Jupyter notebooks are useful for:

- Exploration
- Visualization
- Experimentation
- Model comparison

Production Python modules are used for:

- Automated data collection
- Feature engineering
- Model training
- Evaluation
- Prediction
- API serving

39. Validation and Testing

The system contains validation at multiple stages.

Data validation

Checks include:

- Missing values
- Duplicate timestamps
- Timestamp validity
- Chronological ordering
- Expected schema

Feature validation

Checks include:

- Exactly 70 model features
- Required columns
- Missing feature values
- Target availability
- Chronological order

Model validation

Includes:

- Train/validation/test evaluation
- MAE
- RMSE
- R^2
- Model comparison

Application validation

Includes:

- FastAPI health endpoint
- API error handling
- Dashboard API connectivity
- Forecast retrieval
- SHAP retrieval

40. Limitations

Although the system performs strongly, several limitations remain.

40.1 Location limitation

The current system is primarily designed for Peshawar.

It has not yet been validated as a multi-city Pakistani AQI model.

40.2 Forecast uncertainty

The current forecast provides point predictions.

It does not yet provide formal prediction intervals or uncertainty estimates.

40.3 Recursive error

Because predictions are recursively fed into later predictions, errors can accumulate over 72 hours.

40.4 External data dependency

The system depends on Open-Meteo availability and forecast quality.

If upstream forecast information is inaccurate, the AQI forecast can also be affected.

40.5 Distribution shift

Future air-quality behavior may differ from the historical training data.

Extreme pollution events may therefore produce larger errors.

41. Future Improvements

Several improvements can be added in future versions.

1. Prediction Intervals

Future versions can provide uncertainty ranges around AQI forecasts.

2. Model Drift Monitoring

The system can automatically monitor:

- Feature distribution changes
- AQI distribution changes

- Prediction errors
- Model performance degradation

3. Multi-City Forecasting

The system can be extended to cities such as:

- Islamabad
- Lahore
- Karachi
- Quetta
- Rawalpindi
- Multan

The existing location_id entity in Feast provides a foundation for this extension.

4. Automated Retraining

Retraining can be triggered based on:

- Model performance degradation
- Data drift
- Feature drift
- New historical data

rather than relying only on a fixed schedule.

5. Better Monitoring

A monitoring layer could track:

- Data freshness
- API availability
- Forecast freshness
- Model version
- Prediction latency
- Forecast accuracy

42. Complete Technology Stack

<i>Component</i>	<i>Technology</i>
<i>Programming language</i>	Python
<i>Data processing</i>	pandas, NumPy
<i>Machine learning</i>	scikit-learn

<i>Feature store</i>	Feast
<i>Database</i>	PostgreSQL
<i>Managed database</i>	Supabase
<i>Experiment tracking</i>	MLflow
<i>Model registry</i>	MLflow
<i>Cloud ML platform</i>	DagsHub
<i>Backend</i>	FastAPI
<i>ASGI server</i>	Uvicorn
<i>Dashboard</i>	Streamlit
<i>Visualization</i>	Plotly
<i>Explainability</i>	SHAP
<i>CI/CD</i>	GitHub Actions
<i>Testing</i>	PyTest
<i>Version control</i>	Git/GitHub
<i>Development</i>	VS Code

43. Final Conclusion

Pearls AQI Predictor is a complete machine learning application designed for AQI forecasting in Peshawar.

The project successfully integrates data engineering, feature engineering, time-series machine learning, feature-store management, model registration, recursive forecasting, API development, dashboard development, explainability, and CI/CD.

The system uses approximately two years of hourly historical data and generates 70 engineered model features.

Three machine learning models were compared using chronological evaluation:

- Ridge Regression
- Random Forest Regression
- XGBoost Regression

XGBoost achieved the best documented test performance:

MAE = 1.3132

RMSE = 2.4022

R² = 0.9943

It was therefore selected as the champion model and registered as:

Pearls_AQI_XGBoost@champion

The production forecasting pipeline uses 96 hours of historical context and generates 72 recursive hourly predictions. These predictions are then aggregated into three daily AQI averages for dashboard presentation.

The system uses Feast for feature management, PostgreSQL/Supabase for cloud feature infrastructure, MLflow/DagsHub for model management, FastAPI for backend serving, Streamlit for dashboard, SHAP for explainability, and GitHub Actions for automation.

44. Project Summary

Project: Pearls AQI Predictor

Purpose: AQI monitoring and 3-day forecasting

Location: Peshawar, Pakistan

Historical Data: Approximately 2 years

Raw Records: 17,544

Processed Records: 17,471

Model Features: 70

Prediction Target: Next-hour AQI

Models: Ridge, Random Forest, XGBoost

Champion: XGBoost

Test MAE: 1.3132

Test RMSE: 2.4022

Test R²: 0.9943

Feature Store: Feast

Database: PostgreSQL/Supabase

Model Registry: MLflow/DagsHub

Forecast: Recursive 72 hours

Backend: FastAPI

Frontend: Streamlit

Explainability: SHAP

Automation: GitHub Actions

Repository: Iqrajan2024/AQI_Predictor

Deployment: Streamlit Cloud + FastAPI Cloud